JavaScript is disabled on your browser.

compact1, compact2, compact3

java.time.chrono

Interface ChronoLocalDateTime<D extends ChronoLocalDate>

Type Parameters:

`D - the concrete type for the date of this date-time`

All Superinterfaces:

`Comparable<ChronoLocalDateTime<?>>, Temporal, TemporalAccessor, TemporalAdjuster`

All Known Implementing Classes:

`LocalDateTime`

```
public interface ChronoLocalDateTime<D extends ChronoLocalDate>
extends Temporal, TemporalAdjuster, Comparable<ChronoLocalDateTime<?>>
```

A date-time without a time-zone in an arbitrary chronology, intended for advanced globalization use cases.

Most applications should declare method signatures, fields and variables as `LocalDateTime`, not this interface.

A `ChronoLocalDateTime` is the abstract representation of a local date-time where the `Chronology chronology`, or calendar system, is pluggable. The date-time is defined in terms of fields expressed by `TemporalField`, where most common implementations are defined in `ChronoField`. The chronology defines how the calendar system operates and the meaning of the standard fields.

When to use this interface

The design of the API encourages the use of `LocalDateTime` rather than this interface, even in the case where the application needs to deal with multiple calendar systems. The rationale for this is explored in detail in `ChronoLocalDate`.

Ensure that the discussion in ChronoLocalDate has been read and understood before using this interface.

Implementation Requirements:

This interface must be implemented with care to ensure other classes operate correctly. All implementations that can be instantiated must be final, immutable and thread-safe. Subclasses should be Serializable wherever possible.

Since:

1.8

Method Summary

Default M

Modifier and Type	Method	D
default Temporal	adjustInto(Temporal temporal)	A te ha ar
ChronoZonedDateTime<D>	atZone(ZoneId zone)	C w cr C
default int	compareTo(ChronoLocalDateTime<?> other)	C ti ti cl
boolean	equals(Object obj)	C ti ar in cl
default String	format(DateTimeFormatter formatter)	F us fo
static ChronoLocalDateTime<?>	from(TemporalAccessor temporal)	O C fr ol
default Chronology	getChronology()	

		G th
int	**hashCode**()	A da
default boolean	**isAfter**(**ChronoLocalDateTime**<? > other)	C ti sp ig cl
default boolean	**isBefore**(**ChronoLocalDateTime**<? > other)	C ti sp ig cl
default boolean	**isEqual**(**ChronoLocalDateTime**<? > other)	C ti sp ig cl
boolean	**isSupported**(**TemporalField** field)	C fi
default boolean	**isSupported**(**TemporalUnit** unit)	C u
default **ChronoLocalDateTime**<D>	**minus**(long amountToSubtract, **TemporalUnit** unit)	R th ol sp su
default **ChronoLocalDateTime**<D>	**minus**(**TemporalAmount** amount)	R th ol su
ChronoLocalDateTime<D>	**plus**(long amountToAdd, **TemporalUnit** unit)	R th ol sp

default **ChronoLocalDateTime**<**D**>	**plus**(**TemporalAmount** amount)	R th ol ac
default <R> R	**query**(**TemporalQuery**<R> query)	Q us qu
static **Comparator**<**ChronoLocalDateTime**<?>>	**timeLineOrder**()	G co Ch in ig cl
default long	**toEpochSecond**(**ZoneOffset** offset)	C to se ep 1
default **Instant**	**toInstant**(**ZoneOffset** offset)	C to
D	**toLocalDate**()	G pa
LocalTime	**toLocalTime**()	G pa
String	**toString**()	O as
default **ChronoLocalDateTime**<**D**>	**with**(**TemporalAdjuster** adjuster)	R ol as ac
ChronoLocalDateTime<**D**>	**with**(**TemporalField** field, long newValue)	R th ol sp

Methods inherited from interface java.time.temporal.Temporal

until

Methods inherited from interface java.time.temporal.TemporalAccessor

`get`, `getLong`, `range`

Method Detail

timeLineOrder

```
static Comparator<ChronoLocalDateTime<?>> timeLineOrder()
```

Gets a comparator that compares `ChronoLocalDateTime` in time-line order ignoring the chronology.

This comparator differs from the comparison in `compareTo(java.time.chrono.ChronoLocalDateTime<?>)` in that it only compares the underlying date-time and not the chronology. This allows dates in different calendar systems to be compared based on the position of the date-time on the local time-line. The underlying comparison is equivalent to comparing the epoch-day and nano-of-day.

Returns:

a comparator that compares in time-line order ignoring the chronology

See Also:

`isAfter(java.time.chrono.ChronoLocalDateTime<?>)`, `isBefore(java.time.chrono.ChronoLocalDateTime<?>)`, `isEqual(java.time.chrono.ChronoLocalDateTime<?>)`

from

```
static ChronoLocalDateTime<?> from(TemporalAccessor temporal)
```

Obtains an instance of `ChronoLocalDateTime` from a temporal object.

This obtains a local date-time based on the specified temporal. A `TemporalAccessor` represents an arbitrary set of date and time information, which this factory converts to an instance of `ChronoLocalDateTime`.

The conversion extracts and combines the chronology and the date-time from the temporal object. The behavior is equivalent to using `Chronology.localDateTime(TemporalAccessor)` with the extracted chronology. Implementations are permitted to perform optimizations such as accessing those fields that are equivalent to the relevant objects.

This method matches the signature of the functional interface `TemporalQuery` allowing it to be used as a query via method reference, `ChronoLocalDateTime::from`.

Parameters:

`temporal - the temporal object to convert, not null`

Returns:

`the date-time, not null`

Throws:

`DateTimeException - if unable to convert to a ChronoLocalDateTime`

See Also:

`Chronology.localDateTime(TemporalAccessor)`

getChronology

```
default Chronology getChronology()
```

Gets the chronology of this date-time.

The `Chronology` represents the calendar system in use. The era and other fields in `ChronoField` are defined by the chronology.

Returns:

`the chronology, not null`

toLocalDate

```
D toLocalDate()
```

Gets the local date part of this date-time.

This returns a local date with the same year, month and day as this date-time.

Returns:

`the date part of this date-time, not null`

toLocalTime

```
LocalTime toLocalTime()
```

Gets the local time part of this date-time.

This returns a local time with the same hour, minute, second and nanosecond as this date-time.

Returns:

the time part of this date-time, not null

isSupported

`boolean isSupported(TemporalField field)`

Checks if the specified field is supported.

This checks if the specified field can be queried on this date-time. If false, then calling the `range`, `get` and `with(TemporalField, long)` methods will throw an exception.

The set of supported fields is defined by the chronology and normally includes all `ChronoField` date and time fields.

If the field is not a `ChronoField`, then the result of this method is obtained by invoking `TemporalField.isSupportedBy(TemporalAccessor)` passing `this` as the argument. Whether the field is supported is determined by the field.

Specified by:

`isSupported` in interface `TemporalAccessor`

Parameters:

`field - the field to check, null returns false`

Returns:

`true if the field can be queried, false if not`

isSupported

`default boolean isSupported(TemporalUnit unit)`

Checks if the specified unit is supported.

This checks if the specified unit can be added to or subtracted from this date-time. If false, then calling the `plus(long, TemporalUnit)` and `minus` methods will throw an exception.

The set of supported units is defined by the chronology and normally includes all `ChronoUnit` units except `FOREVER`.

If the unit is not a `ChronoUnit`, then the result of this method is obtained by invoking `TemporalUnit.isSupportedBy(Temporal)` passing `this` as the argument. Whether the unit is supported is determined by the unit.

Specified by:

`isSupported` in interface `Temporal`

Parameters:

unit - the unit to check, null returns false

Returns:

true if the unit can be added/subtracted, false if not

with

default ChronoLocalDateTime<D> with(TemporalAdjuster adjuster)

Returns an adjusted object of the same type as this object with the adjustment made.

This adjusts this date-time according to the rules of the specified adjuster. A simple adjuster might simply set the one of the fields, such as the year field. A more complex adjuster might set the date to the last day of the month. A selection of common adjustments is provided in TemporalAdjusters. These include finding the "last day of the month" and "next Wednesday". The adjuster is responsible for handling special cases, such as the varying lengths of month and leap years.

Some example code indicating how and why this method is used:

```
  date = date.with(Month.JULY);        // most key classes implement TemporalAdju
  date = date.with(lastDayOfMonth());  // static import from Adjusters
  date = date.with(next(WEDNESDAY));   // static import from Adjusters and DayOfW
```

Specified by:

with in interface Temporal

Parameters:

adjuster - the adjuster to use, not null

Returns:

an object of the same type with the specified adjustment made, not null

Throws:

DateTimeException - if unable to make the adjustment

ArithmeticException - if numeric overflow occurs

with

ChronoLocalDateTime<D> with(TemporalField field,
                            long newValue)

Returns an object of the same type as this object with the specified field altered.

This returns a new object based on this one with the value for the specified field changed. For example, on a `LocalDate`, this could be used to set the year, month or day-of-month. The returned object will have the same observable type as this object.

In some cases, changing a field is not fully defined. For example, if the target object is a date representing the 31st January, then changing the month to February would be unclear. In cases like this, the field is responsible for resolving the result. Typically it will choose the previous valid date, which would be the last valid day of February in this example.

Specified by:

`with` in interface `Temporal`

Parameters:

`field - the field to set in the result, not null`

`newValue - the new value of the field in the result`

Returns:

`an object of the same type with the specified field set, not null`

Throws:

`DateTimeException - if the field cannot be set`

`ArithmeticException - if numeric overflow occurs`

plus

```
default ChronoLocalDateTime<D> plus(TemporalAmount amount)
```

Returns an object of the same type as this object with an amount added.

This adjusts this temporal, adding according to the rules of the specified amount. The amount is typically a `Period` but may be any other type implementing the `TemporalAmount` interface, such as `Duration`.

Some example code indicating how and why this method is used:

```
  date = date.plus(period);               // add a Period instance
  date = date.plus(duration);             // add a Duration instance
  date = date.plus(workingDays(6));       // example user-written workingDays me
```

Note that calling `plus` followed by `minus` is not guaranteed to return the same date-time.

Specified by:

`plus` in interface `Temporal`

Parameters:

`amount - the amount to add, not null`

Returns:

an object of the same type with the specified adjustment made, not null

Throws:

DateTimeException - if the addition cannot be made

ArithmeticException - if numeric overflow occurs

plus

```
ChronoLocalDateTime<D> plus(long amountToAdd,
                            TemporalUnit unit)
```

Returns an object of the same type as this object with the specified period added.

This method returns a new object based on this one with the specified period added. For example, on a `LocalDate`, this could be used to add a number of years, months or days. The returned object will have the same observable type as this object.

In some cases, changing a field is not fully defined. For example, if the target object is a date representing the 31st January, then adding one month would be unclear. In cases like this, the field is responsible for resolving the result. Typically it will choose the previous valid date, which would be the last valid day of February in this example.

Specified by:

plus in interface Temporal

Parameters:

amountToAdd - the amount of the specified unit to add, may be negative

unit - the unit of the amount to add, not null

Returns:

an object of the same type with the specified period added, not null

Throws:

DateTimeException - if the unit cannot be added

ArithmeticException - if numeric overflow occurs

minus

```
default ChronoLocalDateTime<D> minus(TemporalAmount amount)
```

Returns an object of the same type as this object with an amount subtracted.

This adjusts this temporal, subtracting according to the rules of the specified amount. The amount is typically a `Period` but may be any other type implementing the `TemporalAmount` interface, such as `Duration`.

Some example code indicating how and why this method is used:

```
  date = date.minus(period);               // subtract a Period instance
  date = date.minus(duration);             // subtract a Duration instance
  date = date.minus(workingDays(6));       // example user-written workingDays me
```

Note that calling `plus` followed by `minus` is not guaranteed to return the same date-time.

Specified by:

`minus` in interface `Temporal`

Parameters:

`amount` - the amount to subtract, not null

Returns:

an object of the same type with the specified adjustment made, not null

Throws:

`DateTimeException` - if the subtraction cannot be made

`ArithmeticException` - if numeric overflow occurs

minus

```
default ChronoLocalDateTime<D> minus(long amountToSubtract,
                                     TemporalUnit unit)
```

Returns an object of the same type as this object with the specified period subtracted.

This method returns a new object based on this one with the specified period subtracted. For example, on a `LocalDate`, this could be used to subtract a number of years, months or days. The returned object will have the same observable type as this object.

In some cases, changing a field is not fully defined. For example, if the target object is a date representing the 31st March, then subtracting one month would be unclear. In cases like this, the field is responsible for resolving the result. Typically it will choose the previous valid date, which would be the last valid day of February in this example.

Specified by:

`minus` in interface `Temporal`

Parameters:

amountToSubtract - the amount of the specified unit to subtract, may be negative

unit - the unit of the amount to subtract, not null

Returns:

an object of the same type with the specified period subtracted, not null

Throws:

DateTimeException - if the unit cannot be subtracted

ArithmeticException - if numeric overflow occurs

query

```
default <R> R query(TemporalQuery<R> query)
```

Queries this date-time using the specified query.

This queries this date-time using the specified query strategy object. The `TemporalQuery` object defines the logic to be used to obtain the result. Read the documentation of the query to understand what the result of this method will be.

The result of this method is obtained by invoking the `TemporalQuery.queryFrom(TemporalAccessor)` method on the specified query passing `this` as the argument.

Specified by:

query in interface TemporalAccessor

Type Parameters:

R - the type of the result

Parameters:

query - the query to invoke, not null

Returns:

the query result, null may be returned (defined by the query)

Throws:

DateTimeException - if unable to query (defined by the query)

ArithmeticException - if numeric overflow occurs (defined by the query)

adjustInto

`default Temporal adjustInto(Temporal temporal)`

Adjusts the specified temporal object to have the same date and time as this object.

This returns a temporal object of the same observable type as the input with the date and time changed to be the same as this.

The adjustment is equivalent to using `Temporal.with(TemporalField, long)` twice, passing `ChronoField.EPOCH_DAY` and `ChronoField.NANO_OF_DAY` as the fields.

In most cases, it is clearer to reverse the calling pattern by using `Temporal.with(TemporalAdjuster)`:

```
   // these two lines are equivalent, but the second approach is recommended
   temporal = thisLocalDateTime.adjustInto(temporal);
   temporal = temporal.with(thisLocalDateTime);
```

This instance is immutable and unaffected by this method call.

Specified by:

`adjustInto` in interface `TemporalAdjuster`

Parameters:

`temporal` - the target object to be adjusted, not null

Returns:

the adjusted object, not null

Throws:

`DateTimeException` - if unable to make the adjustment

`ArithmeticException` - if numeric overflow occurs

format

`default String format(DateTimeFormatter formatter)`

Formats this date-time using the specified formatter.

This date-time will be passed to the formatter to produce a string.

The default implementation must behave as follows:

```
  return formatter.format(this);
```

Parameters:

formatter - the formatter to use, not null

Returns:

the formatted date-time string, not null

Throws:

DateTimeException - if an error occurs during printing

atZone

ChronoZonedDateTime<D> atZone(ZoneId zone)

Combines this time with a time-zone to create a ChronoZonedDateTime.

This returns a ChronoZonedDateTime formed from this date-time at the specified time-zone. The result will match this date-time as closely as possible. Time-zone rules, such as daylight savings, mean that not every local date-time is valid for the specified zone, thus the local date-time may be adjusted.

The local date-time is resolved to a single instant on the time-line. This is achieved by finding a valid offset from UTC/Greenwich for the local date-time as defined by the rules of the zone ID.

In most cases, there is only one valid offset for a local date-time. In the case of an overlap, where clocks are set back, there are two valid offsets. This method uses the earlier offset typically corresponding to "summer".

In the case of a gap, where clocks jump forward, there is no valid offset. Instead, the local date-time is adjusted to be later by the length of the gap. For a typical one hour daylight savings change, the local date-time will be moved one hour later into the offset typically corresponding to "summer".

To obtain the later offset during an overlap, call ChronoZonedDateTime.withLaterOffsetAtOverlap() on the result of this method.

Parameters:

zone - the time-zone to use, not null

Returns:

the zoned date-time formed from this date-time, not null

toInstant

default Instant toInstant(ZoneOffset offset)

Converts this date-time to an Instant.

This combines this local date-time and the specified offset to form an `Instant`.

This default implementation calculates from the epoch-day of the date and the second-of-day of the time.

Parameters:

`offset - the offset to use for the conversion, not null`

Returns:

`an Instant representing the same instant, not null`

toEpochSecond

```
default long toEpochSecond(ZoneOffset offset)
```

Converts this date-time to the number of seconds from the epoch of 1970-01-01T00:00:00Z.

This combines this local date-time and the specified offset to calculate the epoch-second value, which is the number of elapsed seconds from 1970-01-01T00:00:00Z. Instants on the time-line after the epoch are positive, earlier are negative.

This default implementation calculates from the epoch-day of the date and the second-of-day of the time.

Parameters:

`offset - the offset to use for the conversion, not null`

Returns:

`the number of seconds from the epoch of 1970-01-01T00:00:00Z`

compareTo

```
default int compareTo(ChronoLocalDateTime<?> other)
```

Compares this date-time to another date-time, including the chronology.

The comparison is based first on the underlying time-line date-time, then on the chronology. It is "consistent with equals", as defined by `Comparable`.

For example, the following is the comparator order:

1. `2012-12-03T12:00 (ISO)`
2. `2012-12-04T12:00 (ISO)`
3. `2555-12-04T12:00 (ThaiBuddhist)`
4. `2012-12-05T12:00 (ISO)`

Values #2 and #3 represent the same date-time on the time-line. When two values represent the same date-time, the chronology ID is compared to

distinguish them. This step is needed to make the ordering "consistent with equals".

If all the date-time objects being compared are in the same chronology, then the additional chronology stage is not required and only the local date-time is used.

This default implementation performs the comparison defined above.

Specified by:

`compareTo` in interface `Comparable<ChronoLocalDateTime<?>>`

Parameters:

`other - the other date-time to compare to, not null`

Returns:

`the comparator value, negative if less, positive if greater`

isAfter

```
default boolean isAfter(ChronoLocalDateTime<?> other)
```

Checks if this date-time is after the specified date-time ignoring the chronology.

This method differs from the comparison in `compareTo(java.time.chrono.ChronoLocalDateTime<?>)` in that it only compares the underlying date-time and not the chronology. This allows dates in
different calendar systems to be compared based on the time-line position.

This default implementation performs the comparison based on the epoch-day and nano-of-day.

Parameters:

`other - the other date-time to compare to, not null`

Returns:

`true if this is after the specified date-time`

isBefore

```
default boolean isBefore(ChronoLocalDateTime<?> other)
```

Checks if this date-time is before the specified date-time ignoring the chronology.

This method differs from the comparison in `compareTo(java.time.chrono.ChronoLocalDateTime<?>)` in that it only compares the underlying date-time and not the chronology. This allows dates

in
different calendar systems to be compared based on the time-line position.

This default implementation performs the comparison based on the epoch-day and nano-of-day.

Parameters:

`other - the other date-time to compare to, not null`

Returns:

`true if this is before the specified date-time`

isEqual

```
default boolean isEqual(ChronoLocalDateTime<?> other)
```

Checks if this date-time is equal to the specified date-time ignoring the chronology.

This method differs from the comparison in `compareTo(java.time.chrono.ChronoLocalDateTime<?>)` in that it only compares the underlying date and time and not the chronology. This allows date-times in different calendar systems to be compared based on the time-line position.

This default implementation performs the comparison based on the epoch-day and nano-of-day.

Parameters:

`other - the other date-time to compare to, not null`

Returns:

`true if the underlying date-time is equal to the specified date-time on the timeline`

equals

```
boolean equals(Object obj)
```

Checks if this date-time is equal to another date-time, including the chronology.

Compares this date-time with another ensuring that the date-time and chronology are the same.

Overrides:

`equals in class Object`

Parameters:

obj - the object to check, null returns false

Returns:

true if this is equal to the other date

See Also:

Object.hashCode(), HashMap

hashCode

int hashCode()

A hash code for this date-time.

Overrides:

hashCode in class Object

Returns:

a suitable hash code

See Also:

Object.equals(java.lang.Object), System.identityHashCode(java.lang.Object)

toString

String toString()

Outputs this date-time as a String.

The output will include the full local date-time.

Overrides:

toString in class Object

Returns:

```
a string representation of this date-time, not null
```


Submit a bug or feature
For further API reference and developer documentation, see Java SE Documentation. That documentation contains more detailed, developer-targeted descriptions, with conceptual overviews, definitions of terms, workarounds, and working code examples.
Copyright © 1993, 2026, Oracle and/or its affiliates. All rights reserved. Use is subject to license terms.
Also see the documentation redistribution policy.

Scripting on this page tracks web page traffic, but does not change the content in any way.